# Open-Source Implementation of Deep Research with LangGraph

Hongru Chen
@columbia.edu

Mei Yue
my2903@columbia.edu

Xiaote Liu
@columbia.edu

JY
@columbia.edu

July 28, 2025

**Abstract**

This report introduces LegalDeepResearch, an open-source system for AI-driven, multi-step legal document analysis using LangGraph, retrieval-augmented generation (RAG), and domain-adapted legal models. The system performs in-depth research by segmenting long legal texts, retrieving relevant statutes and precedents, and generating structured, citation-aware outputs grounded in legal reasoning. We implement a modular orchestrator-worker architecture with nested graph states to manage tasks such as document summarization, appeal suggestion, and unfair clause detection. The system integrates FAISS-based legal retrieval, LexGLUE-based classifiers, and large language models for generative analysis. A web interface enables users to upload legal PDFs, selectively apply analytical modules, and view or export structured reports. This work demonstrates the feasibility of combining symbolic workflows, retrieval pipelines, and generative AI to support complex legal analysis in research, education, and professional practice.

## 1 Introduction and Project Overview

### 1.1 Background and Motivation

The emergence of powerful large language models (LLMs) has enabled new capabilities for **automating complex cognitive tasks**. One particularly promising application is **"deep research"**—leveraging multiple AI-powered agents that **autonomously** conduct multi-step research by **searching, analyzing,** and **synthesizing** information from various sources to **generate** comprehensive reports.

Several studies have explored such AI-assisted research workflows using LLMs. Nakano et al. [2022] introduced **WebGPT**, an early attempt at training GPT-3 to **search the web** and answer questions. **Chain-of-thought (CoT)** prompting techniques introduced by Wei et al. [2022] have been utilized to improve **complex reasoning** in research tasks. Furthermore, the development of agentic workflows such as **ReAct** introduced by Yao et al. [2023] combines **reasoning and acting** with LLMs, while Ahn et al. [2022]'s **SayCan** grounds their outputs to be **accurate and relevant** to a particular domain (i.e. the physical world). Shinn et al. [2023] introduced the **Reflexion** framework that implements **self-reflection capabilities** to **improve reasoning**. Langchain and subsequently **LangGraph** provide frameworks for implementing complex LLM workflows with state management. More recently, Lu et al. [2024] proposed **AI Scientist**, applying these **agentic workflows** for directing LLMs to generate ideas for **AI research**, produce code to test them, and document the inquiry.

Several commercial platforms have also recently introduced deep research capabilities that have popularized the paradigm. OpenAI's **ChatGPT Deep Research** uses GPT-4 to perform in-depth research with multi-step reasoning and automatic citation [OpenAI, 2025]. Google's **Gemini Deep Research** utilizes Gemini models for real-time information retrieval across various sources [et al, 2024b,a]. **Perplexity AI** offers enhanced research capabilities with its advanced search functionality and recently also released a deep research feature [PerplexityAI, 2025]. These platforms

demonstrate the possibility of autonomous research but lack the customizability, transparency, and cost-effectiveness that open-source alternatives could provide.

### 1.2 Project Objectives

As such, we present **LegalDeepResearch**, an open-source implementation of multi-step legal document analysis using the LangGraph framework and retrieval-augmented generation (RAG). Our system is designed to support legal professionals, researchers, and enterprise users by automating key legal reasoning tasks. Specifically, it addresses the following legal document analysis needs:

- **Legal Document Summarization and Key Element Extraction:** Given a court decision or other legal text, the system automatically extracts basic case metadata (e.g., court, date, cause of action), identifies key conclusions and factual context, and produces structured summaries or reports for downstream use.
- **AI-Assisted Appeal Recommendation:** For uploaded judgments, the system identifies ruling outcomes and risk indicators, and—by retrieving similar precedent cases or legal provisions—generates a first-step AI assessment of whether an appeal might be warranted.
- **Unfair Clause Detection in Contracts (LexGLUE):** The system analyzes uploaded contract files, segments them into sentences, and uses a fine-tuned classifier to detect potentially unfair clauses (e.g., unilateral termination, automatic renewal), highlighting them with explanations and generating standardized audit reports.

These capabilities enable LegalDeepResearch to support use cases in litigation support, internal legal review, and contract risk assessment. The system reduces time spent reading, aids decision-making, and provides traceable, interpretable results across multiple legal tasks.

## 2 Approach and Implementation

LegalDeepResearch adopts a modular architecture that supports multiple legal analysis workflows, triggered by a single user upload. The overall process is built to reflect actual legal reasoning tasks and includes three optional and parallel modules, as shown in Figure **??**. Each module operates either interactively (with user feedback) or automatically (as single-pass pipelines), providing flexibility for legal researchers and practitioners.

### 2.1 Workflow Overview

Upon uploading a PDF document—such as a court decision or a terms-of-service contract—the system parses the input into a structured format. Users are then presented with three analytical options:

1. **Legal Summarization**
2. **Appeal Recommendation Analysis**
3. **Unfair Clause Detection**

These modules can be executed independently or in parallel, depending on the user's needs.

### 2.2 Legal Summarization

The Legal Summarization module is designed for nuanced, human-in-the-loop summarization of judicial opinions or legal filings. After structural parsing (e.g., extracting court name, judgment date, case type), the document is segmented into legal-relevant units such as background, reasoning, and ruling. For each segment:

- The system constructs section-specific search queries based on legal cues and metadata.
- External content is retrieved via Tavily, Google Custom Search (limited to legal domains), and DuckDuckGo.
- Retrieved content is deduplicated and cleaned, then passed into prompt templates for LLM-based content generation.

Users can expand retrieved evidence, modify summaries, or skip sections entirely. The final result is exported in structured formats (Markdown, HTML, PDF), enabling traceable downstream reporting.

### 2.3 Appeal Recommendation Analysis

LegalDeepResearch is composed of three independently deployable pipelines: *legal summarization*, *appeal suggestion*, and *unfair clause detection*. Each pipeline shares the same ingestion and document parsing foundation but diverges in downstream logic and model usage. The modular design allows each component to be updated or replaced without affecting the others.

**Legal Summarization Pipeline.** This pipeline is segment-aware and comprises the following key stages:

- **Segmentation**: Based on PDF layout and semantic cue phrases (e.g., "Judgment", "Facts"), the document is divided into distinct sections.
- **RAG Integration**: For each section, queries are automatically generated using prompt engineering strategies that reflect the section type. For instance, "Court Reasoning" segments emphasize causal chains and legal justification. These queries are sent asynchronously to multiple retrieval backends.
- **Prompt Templates**: Each section uses a custom prompt template optimized for style and expected content structure. These templates are designed to avoid hallucination by including retrieved context inline and specifying the tone (neutral vs. argumentative).
- **User Interactivity**: Results are cached and linked to editable widgets, enabling users to inspect sources, rerun generations, or modify outputs at the segment level.

**Appeal Recommendation Pipeline.** This module operates at the document level and bypasses segmentation. Its core steps include:

- **Document-Level Parsing**: A legal-tuned LLM extracts the ruling, reasoning structure, and any penalties imposed.
- **FAISS-Based Retrieval**: The system encodes case embeddings using Sentence-BERT and performs dense similarity search against a curated corpus of precedents and statutes.
- **Risk Scoring**: Retrieved items are used to populate a multi-dimensional risk model. Each factor (e.g., inconsistent penalty, lack of reasoning) is assigned a heuristic weight, which is interpreted in plain language by the final RAG generation.
- **Output**: The module emits a single report containing appeal-worthiness score, retrieved case anchors, and a breakdown of key risk indicators.

**Unfair Clause Detection Pipeline.** This pipeline focuses on clause classification and explanation generation:

- **Clause Segmentation**: Contracts are segmented into meaningful units using newline breaks and syntactic cues (e.g., semicolons, enumerated points).
- **LexGLUE Classifier**: Each clause is passed through a transformer model fine-tuned on the UNFAIR-ToS dataset. The classifier outputs both the class label and a confidence score.
- **Explanation and Redlining**: For each flagged clause, the system generates a natural language explanation using a generative LLM trained to map classification labels to human-readable summaries. Suggested revisions are also included.
- **Audit View**: All flagged content is highlighted inline within the original contract layout and also compiled into a structured compliance PDF with clickable tags.

This modular pipeline structure ensures extensibility: new modules (e.g., citation recommendation, translation) can be added with minimal disruption to the current codebase, given the shared architecture and LangGraph-based orchestration.

An overview of the workflow is shown in Figure 1, which illustrates how user actions trigger specific analytical modules and how different components operate.

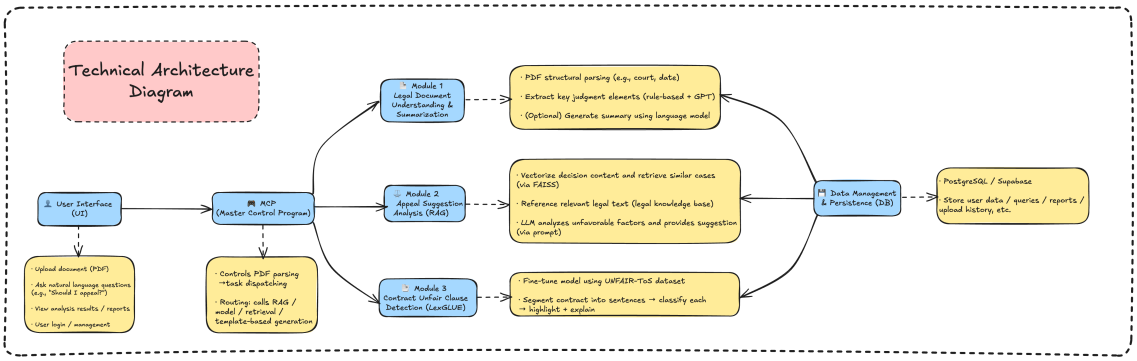



Figure 1: Technical Architecture Diagram: A modular legal AI system comprising a user interface (UI), a master control program (MCP), and three specialized modules: legal document summarization, appeal suggestion via retrieval-augmented generation (RAG), and unfair clause detection using LexGLUE. All components interact with a persistent data layer (PostgreSQL/Supabase) for storing user queries, reports, and upload history. The system enables PDF ingestion, natural language queries, legal reasoning, and structured output.

## 2.4 Unfair Clause Detection

Targeting contract review and compliance auditing, this module segments uploaded contracts into clause-level units. Each clause is processed by a fine-tuned classifier trained on the UNFAIR-ToS dataset from LexGLUE. The system:

- Flags clauses with high risk of unfairness
- Categorizes each by type (e.g., unilateral termination)
- Provides plain-language explanations and revision suggestions

Flagged clauses are rendered in an interactive audit viewer or compiled into a downloadable compliance report.

## 2.5 Tech Stack (Tools/Packages) & Implementation Details

Using Python as our primary programming language, we built the LegalDeepResearch platform on the following core technologies—prioritizing modularity, interpretability, and task-specific extensibility:

- **LangGraph** for orchestrating multi-step legal workflows and managing segment-level document state
- **FAISS** for fast similarity search over legal precedent and statute embeddings
- **Hugging Face** Transformers for deploying fine-tuned legal models, including those trained on the LexGLUE UNFAIR-ToS dataset
- **Next.js and TailwindCSS** for a professional-grade, modular user interface
- **PyMuPDF** for PDF parsing and layout-aware document segmentation
- **Tavily, Google Custom Search, and law-specific engines** for external legal content retrieval
- **Pydantic** for schema validation and structured task payloads
- **AsyncIO** for managing concurrent document processing tasks and external queries

Detailed usage and implementation of each component are described in the following sections.

# 3 User Workflow

The **LegalDeepResearch** platform is designed to accommodate both interactive and automated legal analysis through a modular, user-centered workflow. The process begins with users accessing a streamlined web interface where they can upload one or more legal documents (e.g., court decisions, contracts) and select any combination of available analysis modules.

Upon document upload, the **Master Control Program (MCP)** handles parsing and routing based on user-selected modules. The platform supports three main functionalities:

## 3.1 Legal Document Summarization (Module 1)

For court decisions and judicial documents, the system offers a structured, multi-step summarization workflow. Uploaded documents are parsed into logical sections (e.g., parties, facts, reasoning, judgment), and each section is processed through tailored prompts combined with external legal retrieval:

- **Tavily** for AI-enhanced web search,
- **Google Custom Search** restricted to legal domains (e.g., `law.cornell.edu`),
- **DuckDuckGo** for privacy-respecting keyword search.

Users can expand each segment to inspect extracted facts and sources, regenerate outputs, or skip steps before exporting the final report.

## 3.2 Appeal Recommendation Analysis (Module 2)

When this module is enabled, the system performs a single-pass analysis on the entire legal judgment to evaluate whether the case outcome warrants appeal. It:

- Identifies key outcomes, penalties, and reasoning structures;
- Matches extracted elements against statutory appeal criteria and precedent-based indicators;
- Outputs a structured brief summarizing unfavorable rulings and appeal signals.

This module is fully automated and does not require user interaction during analysis.

## 3.3 Unfair Clause Detection in Contracts (Module 3)

For uploaded contracts or terms of service, the system segments text into clauses and runs each clause through a fine-tuned classifier trained on the **UNFAIR-ToS** dataset from the LexGLUE benchmark. The resulting analysis:

- Flags legally risky or potentially unfair clauses (e.g., unilateral termination, automatic renewal),
- Maps them to a predefined taxonomy of unfairness types,
- Presents a structured AI audit report for reviewer action.

## 3.4 Hybrid Interaction Model

Users may activate one or more of these modules in any order. All analytical results—including summaries, appeal briefs, and unfairness reports—are stored and accessible via the user dashboard. The system's hybrid design supports both:

- **Interactive, iterative research** (Module 1), and
- **Automated, single-pass decision support** (Modules 2 and 3).

An overview of this workflow is shown in Figure 3.4, which illustrates how user actions trigger specific analytical modules and how different components operate.

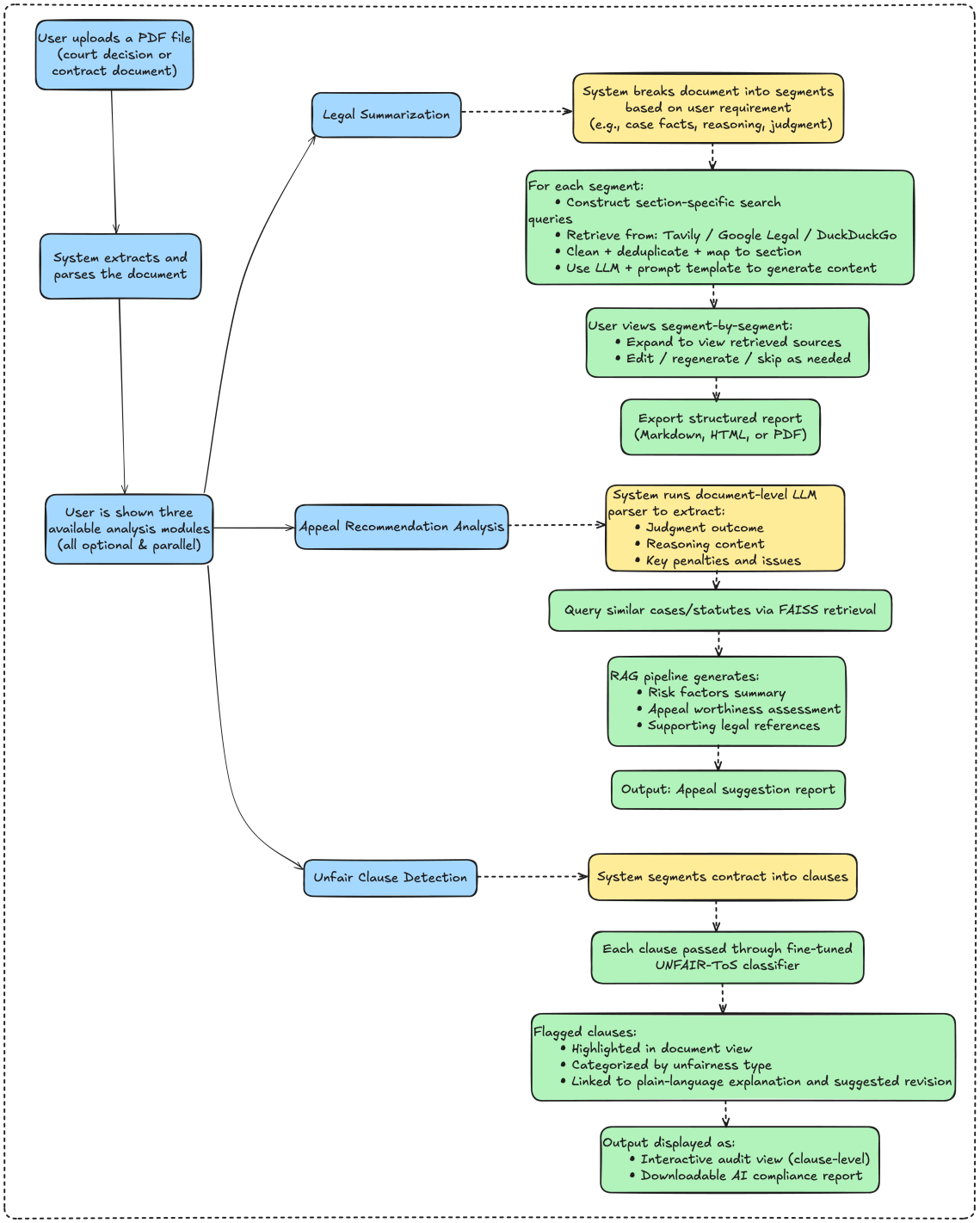



Figure 2: End-to-End LegalDeepResearch Workflow Diagram: A user-centered interaction pipeline supporting legal document analysis through three specialized modules: Legal Summarization, Appeal Recommendation Analysis, and Unfair Clause Detection. Upon uploading a court decision or contract document, the system parses and processes the content, then presents users with optional analysis paths. The summarization module enables section-specific retrieval, LLM-assisted generation, and user-editable reports. The appeal module uses a document-level parser and FAISS retrieval to assess appeal risks. The unfair clause module applies a fine-tuned LexGLUE classifier to highlight problematic contract terms. All outputs are presented in structured, reviewable formats to support both legal research and compliance workflows.

# 4 Conclusion

LegalDeepResearch presents a modular, AI-powered system for automated legal document analysis across multiple task types, including structured summarization, appeal recommendation, and unfair clause detection. By combining segment-level document processing, retrieval-augmented generation (RAG), and fine-tuned legal models, the platform supports both interactive research workflows and fully automated decision support. Its hybrid design allows users to flexibly engage with different modules based on document type and research goals, while maintaining transparency, traceability, and extensibility.

The system demonstrates the feasibility of applying large language models and symbolic workflow control (via LangGraph) to real-world legal tasks, from litigation review to contract compliance analysis. Through the integration of external search providers, local vector search, and domain-adapted classification models, LegalDeepResearch offers a robust foundation for scalable and interpretable legal AI applications.

Future work will focus on evaluating model performance across jurisdictions, expanding the legal retrieval corpus, and incorporating user feedback into iterative refinement loops. We also plan to explore deeper integration with legal knowledge graphs, court databases, and document editing workflows to further enhance real-world utility and adoption.

# References


M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL https://arxiv.org/abs/2204.01691.

G. T. et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024a. URL https://arxiv.org/abs/2403.05530.

G. T. et al. Gemini: A family of highly capable multimodal models, 2024b. URL https://arxiv.org/abs/2312.11805.

C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. URL https://arxiv.org/abs/2408.06292.

R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman. Webgpt: Browser-assisted question-answering with human feedback, 2022. URL https://arxiv.org/abs/2112.09332.

OpenAI. Introducing deep research, 2025. URL https://openai.com/index/introducing-deep-research.

PerplexityAI, 2025. URL https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.